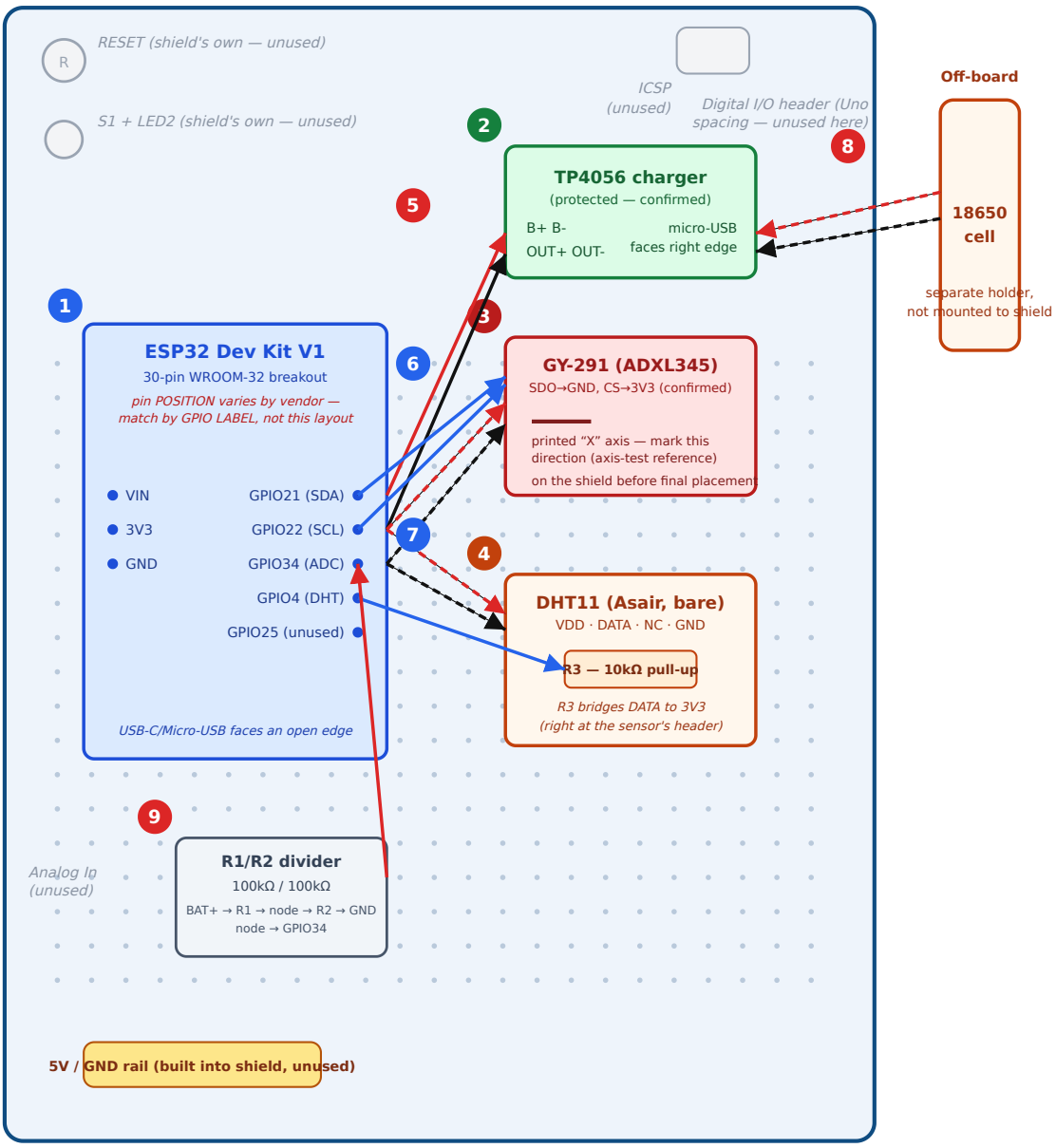
ESP32 Dev Kit V1 Bench Rig — Prototype Shield Build Guide

Temp + Gravity module bench prototype: ESP32 Dev Kit V1 + TP4056 charger + GY-291 (ADXL345) + DHT11 (bare, Asair) mounted on an Arduino-style Prototype Shield v.5.

This shield does not plug into the Dev Kit V1. It's Arduino Uno-shaped with Uno-spaced headers — a completely different pin pitch from the Dev Kit V1's own 30-pin layout. Its Digital I/O / Analog In / ICSP headers go unused here. What's actually useful is its perfboard prototyping area, which you populate and wire by hand.

> **A 30-pin Dev Kit V1 is noticeably bigger than the ESP8266 sibling rig's D1 Mini** — expect a tighter fit on the shield's perfboard than that build had. Dry-fit the board before soldering anything down.



Prototype Shield v.5 (perfboard side up)

Legend

- Power (3V3 / VIN / BAT+)
- Ground
- Signal (I2C / DHT data)

Before you start — already confirmed on this hardware

- ✅ TP4056 breakout is **protected** (3 ICs: TP4056 + DW01A + FS8205A) — carried over from the ESP8266 rig if it's the same physical board; re-verify if not (see `architecture/design_risks.md`)

- ✅ GY-291 `SDO` / `CS` are dedicated pins, not internally tied — confirmed by photo (silkscreen `GND VCC CS INT1 INT2 SDO SDA SCL`), printed X/Y/Z axis diagram present
- ✅ DHT11 is the **bare 4-pin Asair sensor** (no PCB, no onboard pull-up) — confirmed by photo. Pin order left-to-right: `VDD DATA NC GND`. `R3` (10kΩ) supplies the pull-up this variant needs.
- ⚠️ **ESP32 Dev Kit V1 pin layout varies by manufacturer.** Locate `VIN` / `3V3` / `GND` / `GPIO21` / `GPIO22` / `GPIO34` / `GPIO4` by their printed GPIO labels on your specific board — not by counting header position from the diagram above, which shows a simplified/logical view for exactly this reason. See `architecture/design_risks.md`.

Step-by-step assembly

1. **Dry-fit the Dev Kit V1 first.** Before soldering, lay the board on the shield's perfboard to confirm it actually fits your shield's usable area and pick an orientation with the USB port reachable from an open edge. If it doesn't fit flat, standoffs/dead-bug mounting on the underside is the usual workaround for a full-size Dev Kit V1 on an Uno-shaped shield.
2. **Mount the ESP32 Dev Kit V1.** Push its header pins into the perfboard grid and solder from underneath. Orient it so the USB-C/Micro-USB port faces an open edge — you'll want it accessible for reflashing.
3. **Mount the TP4056 charger breakout.** Position it near a board edge so its micro-USB charging port stays reachable from outside the finished assembly. Tack it down (hot glue or double-sided tape is fine — it doesn't need to be soldered flat, just held still).
4. **Mount the GY-291 and mark its axis.** Position it away from the charger section (keeps switching noise off the I2C lines). **Before securing it, note which way the board's printed "X" axis arrow points** — mark this on the shield itself (a sharpie arrow works). This is your reference for the firmware's axis-calibration step later; repositioning the board after calibration means redoing that step.
5. **Mount the DHT11 and its pull-up.** The bare sensor has four pins in a row (`VDD DATA NC GND`) — solder `R3` (10kΩ) directly between the `DATA` pin and a `3V3` point right at the sensor's own header, so the pull-up sits close to the source rather than routed separately across the board.
6. **Wire the charger to the Dev Kit V1.** Two short wires: TP4056 `OUT+` → Dev Kit V1 `VIN`, and TP4056 `OUT-` → Dev Kit V1 `GND`. The board's onboard regulator runs fine from the cell voltage directly — no separate LDO needed.
7. **Wire the Dev Kit V1 to the GY-291.** Four wires: Dev Kit V1 `3V3` → GY-291 `VCC`, `GND` → `GND`, `GPIO21` → `SDA`, `GPIO22` → `SCL`. **Also wire GY-291 `SDO` → `GND` and `CS` → `3V3`** (both dedicated pins on this board, not internally tied — confirmed from your photo).
8. **Wire the Dev Kit V1 to the DHT11.** Three wires: Dev Kit V1 `3V3` → DHT11 `VDD`, `GND` → `GND`, `GPIO4` → DHT11 `DATA` (the same node `R3`'s pull-up lands on). Leave `NC` unconnected.
9. **Wire the battery.** 18650 cell (in its own holder, off the shield) → TP4056 `B+` / `B-`. Keep the holder separate rather than gluing it down — you'll be swapping/recharging it often during calibration.
10. **Wire the battery-sense divider.** `R1` (100kΩ) from `BAT+` to a node, `R2` (100kΩ) from that node to `GND`, and the same node to Dev Kit V1 `GPIO34`. Single-stage — no onboard-divider quirk to work around here, unlike the ESP8266 rig.

Steps 6-10 can be done in any order — the numbering matches the diagram's callouts for cross-reference, not a strict sequence. Just avoid powering anything on until every wire in a given connection is in place (e.g. don't connect the battery with only one of B+/B- wired).

Power-on and first boot

11. **First power-up: USB only, no battery yet.** Plug the Dev Kit V1's own USB port into your computer. Open a serial monitor at 115200 baud. You should see boot messages — if nothing prints, check the USB cable isn't charge-only and that you selected the right serial port.
12. **Flash the firmware.** Open `firmware/fermentation_tilt_v1_esp32_devkit/fermentation_tilt_v1_esp32_devkit.ino`, fill in the CONFIG section (WiFi, Supabase URL/key, device ID/secret), select board **"DOIT ESP32 DEVKIT V1"** in the Arduino IDE, and upload. Install the **"DHT sensor library"** (Adafruit) + **"Adafruit Unified Sensor"** dependency first if you haven't.
13. **Run the axis test.** Hold the assembled rig upright (GY-291's marked X-axis vertical) — the serial log should print a tilt near 0°. Lay it flat — near 90°. If it's inverted or pinned near 90° in both positions, the firmware's axis assumption doesn't match your mounting; swap which axis `readTiltDegrees()` treats as the long one (comment in the code marks the line). Do this before any calibration data collection — a wrong axis makes every later reading garbage.
14. **Check the DHT11 reads cleanly.** Watch for `Temperature: X.XX C` and `Humidity: X.X%` in the serial log each cycle. A failed read (`Error: could not read DHT11...`) with everything wired per the table below almost always means `R3` isn't actually bridging `DATA` to `3V3` — recheck that joint first.
15. **Connect the battery, confirm the whole cycle posts.** Wire the cell in (step 9), let the unit go through one full sleep/wake cycle (default 900s), and confirm a reading appears on the dashboard with temperature, tilt, battery, and humidity all populated.

Reference tables

Connection	From	To	Notes
Power	TP4056 OUT+	Dev Kit V1 VIN	Same node as BAT+
Ground	TP4056 OUT-	Dev Kit V1 GND	
Power	Dev Kit V1 3V3	GY-291 VCC	
I2C	Dev Kit V1 GPIO21	GY-291 SDA	
I2C	Dev Kit V1 GPIO22	GY-291 SCL	
Address set	GY-291 SDO	GND	I2C address 0x53
Mode set	GY-291 CS	3V3	Forces I2C mode
Power	Dev Kit V1 3V3	DHT11 VDD	
Ground	Dev Kit V1 GND	DHT11 GND	
Data	Dev Kit V1 GPIO4	DHT11 DATA	Pulled up to 3V3 through R3 (10kΩ), at the sensor
ADC	BAT+	Dev Kit V1 GPIO34	Via 100kΩ/100kΩ external divider (R1/R2), single-stage

Part	Qty	Notes
100kΩ resistor	2	Battery divider (R1, R2)
10kΩ resistor	1	DHT11 pull-up (R3) — required, bare sensor has none onboard

18650 cell + holder	1	~2500mAh
Jumper wires	~15	Solid-core preferred for perfboard soldering

See `SPEC.md`, `architecture/net_plan.md`, and `architecture/design_risks.md` for the full electrical derivation and design history behind these connections.